

1886. Determine Whether Matrix Can Be Obtained By Rotation

Solved 

Easy

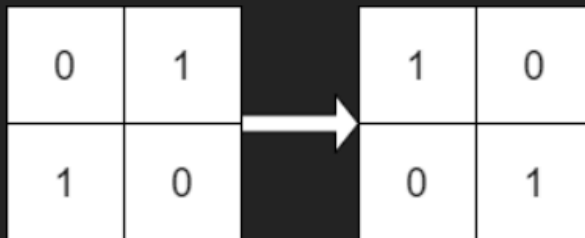
 Topics

 Companies

 Hint

Given two $n \times n$ binary matrices `mat` and `target`, return `true` if it is possible to make `mat` equal to `target` by **rotating** `mat` in **90-degree increments**, or `false` otherwise.

Example 1:



Input: `mat = [[0,1],[1,0]]`, `target = [[1,0],[0,1]]`

Output: `true`

Explanation: We can rotate `mat` 90 degrees clockwise to make `mat` equal `target`.

Example 2:

0	1
1	1

1	0
0	1

Input: `mat = [[0,1],[1,1]]`, `target = [[1,0],[0,1]]`

Output: `false`

Explanation: It is impossible to make `mat` equal to `target` by rotating `mat`.

Example 3:

0	0	0
0	1	0
1	1	1



1	0	0
1	1	0
1	0	0



1	1	1
0	1	0
0	0	0

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`, `target = [[1,1,1],[0,1,0],[0,0,0]]`

Output: `true`

Explanation: We can rotate `mat` 90 degrees clockwise two times to make `mat` equal `target`.

Constraints:

- `n == mat.length == target.length`
- `n == mat[i].length == target[i].length`
- `1 <= n <= 10`
- `mat[i][j]` and `target[i][j]` are either `0` or `1`.

Python:

class Solution:

```

def rotate(self, mat):
    n = len(mat)

    # transpose
    for i in range(n):
        for j in range(i, n):
            mat[i][j], mat[j][i] = mat[j][i], mat[i][j]

    # reverse rows
    for i in range(n):
        mat[i].reverse()

def findRotation(self, mat, target):
    for _ in range(4):
        if mat == target:
            return True
        self.rotate(mat)
    return False

```

JavaScript:

```

var findRotation = function(mat, target) {
    const rotate = (mat) => {
        const n = mat.length;

        // transpose
        for (let i = 0; i < n; i++) {
            for (let j = i; j < n; j++) {
                [mat[i][j], mat[j][i]] = [mat[j][i], mat[i][j]];
            }
        }

        // reverse rows
        for (let i = 0; i < n; i++) {
            mat[i].reverse();
        }
    };

    for (let i = 0; i < 4; i++) {
        if (JSON.stringify(mat) === JSON.stringify(target)) return true;
        rotate(mat);
    }

    return false;
};

```

Java:

```
class Solution {
    public void rotate(int[][] mat) {
        int n = mat.length;

        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int temp = mat[i][j];
                mat[i][j] = mat[j][i];
                mat[j][i] = temp;
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n / 2; j++) {
                int temp = mat[i][j];
                mat[i][j] = mat[i][n - j - 1];
                mat[i][n - j - 1] = temp;
            }
        }
    }

    public boolean findRotation(int[][] mat, int[][] target) {
        for (int k = 0; k < 4; k++) {
            if (isEqual(mat, target)) return true;
            rotate(mat);
        }
        return false;
    }

    private boolean isEqual(int[][] a, int[][] b) {
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[0].length; j++) {
                if (a[i][j] != b[i][j]) return false;
            }
        }
        return true;
    }
}
```